Guess Market

Exercise 4 (Bonus) - Web Client

README and Submission Documentation

Submitted By

Name	Nave Kluska
ID	324168160
Email	navekluska@gmail.com

GitHub Repository

https://github.com/NaveKluska/GuessMarket

The web client's source is in the repository's `guess-market-web` folder.

The URL to Open

http://localhost:5173

`run.bat` opens this address in the default browser by itself once the client has started. If it does not, open it manually.

Running the Web Client

The web client is an additional client for the existing Exercise 3 server. The server is started exactly as in Exercise 3 - nothing about Tomcat, the WAR, or how they are started changes.

1. **Start the server, as in Exercise 3:** copy `guessmarket.war` into Tomcat's `webapps` folder and start Tomcat (`bin\startup.bat`). The `guessmarket.war` in this zip is a byte-for-byte copy of the one submitted for Exercise 3, included only so this submission works on its own.
2. **Double-click `web-client\run.bat`.** It checks that Node.js is installed, runs `npm install` (the first run downloads the libraries - about a minute, internet needed), then starts the client.
3. **The browser opens `http://localhost:5173`.** Log in with any new name.
4. **Keep the `run.bat` window open** while using the client - closing it stops the client.

Requirements

Requirement	Notes
Node.js 18, 20, or 22 and newer	`run.bat` checks the installed version and explains if it is missing or too old.
Internet on the first run	Only for `npm install` to download the libraries.
The Exercise 3 server	Tomcat on `localhost:8080` with `guessmarket.war` deployed.
Port 5173 free	The port the web client runs on.

Using It and Troubleshooting

Trying Several Users at Once

Each browser tab is its own logged-in user, so opening another tab at `http://localhost:5173` lets you log in as someone else. The Exercise 3 JavaFX client and the web client can be used side by side against the same server - both show the same data, live.

If Something Goes Wrong

What you see	What to do
"Cannot reach the Guess Market server" / a red "Server unreachable" chip	Tomcat is not running, or `guessmarket.war` is not deployed. Start it - the client reconnects on its own within a second.
`run.bat` says Node.js is missing or too old	Install the current LTS version from nodejs.org and run `run.bat` again.
"Port 5173 is already in use"	Another copy of the web client is already running - close its window first.
`npm install` failed	Check the internet connection and run `run.bat` again.

What the Web Client Includes

The Three Required Screens

- **Login** - same rules as the JavaFX client: the name must be new; a taken name shows the server's error and lets you try again.
- **Events** - same layout as the JavaFX Events tab: method / status / commission filters, the event list, and a read-only view of the selected event (LMSR price cards and trade history; Order Book bids, asks, statistics and participations).
- **Account (the user screen)** - same layout as the JavaFX Account tab: every user with their balance and Market Maker tag; the selected user's balance, blocked state, deposit and account history; every event with that user's participation in it; and trading right there - LMSR buying, Order Book buy/sell orders, and Open / Close for the event's Market Maker. File upload is left out, as the specification allows.

Beyond What Was Required

The specification asks for the three screens above and states that Exercise 3 bonuses are not expected in the web client. Two more Exercise 3 features were ported anyway, since they use the same existing endpoints:

- **Create Event** - the "+ Create Event" button on your own Account page, with the same fields, ranges and defaults as the JavaFX dialog.
- **Chat** - the Exercise 3 chat bonus, as a third tab. Web and JavaFX users share the same conversation.

How It Talks to the Server

A browser only lets a page's JavaScript read responses from the same address the page was loaded from (the "same-origin policy"). The page is served from `localhost:5173` and the server lives at `localhost:8080` - two different addresses - so the browser would block every answer from the server. A server can allow this explicitly (CORS headers), but that would mean changing the server, which this exercise does not allow.

Instead, the small development server that `run.bat` starts (Vite) does two jobs: it serves the web client's files, and it forwards every request under `/guessmarket/...` to `localhost:8080` (three lines in `vite.config.js`). The browser only ever talks to one address, Tomcat receives ordinary requests, and the server stays untouched.

Server API Reference

The web client calls exactly the same endpoints as the JavaFX client, with the same request bodies - every one of them listed below. The only Exercise 3 endpoint it does not use is file upload.

Method	Endpoint	Request Body	Used for
POST	`/users/register`	`{"name": "alice"}`	Login.
POST	`/users/deposit`	`{"userName": "alice", "amount": 100}`	Deposit.
GET	`/users`	-	Users list, balances, account history.
GET	`/events`	-	Event list.
GET	`/events?name=X`	-	One event's full details.
POST	`/events/open`	`{"mmName": "alice", "eventName": "X"}`	MM opens the event.
POST	`/events/close`	`{"mmName": "alice", "eventName": "X", "winningOptionIndex": 0}`	MM closes it, picks the winner.
POST	`/events/buy`	`{"memberName": "alice", "eventName": "X", "optionIndex": 0, "quantity": 5}`	Buy shares (LMSR events).
POST	`/events/order`	`{"userName": "alice", "eventName": "X", "optionIndex": 0, "side": "BUY", "price": 0.45, "quantity": 4}`	Bid/ask (Order Book events).
POST	`/events/create-lmsr`	`{"creatorName": "alice", "name": "X", "description": "...", "commission": 5, "commissionType": "ON_PURCHASE", "optionNames": ["Yes", "No"], "b": 100}`	Create Event (LMSR).
POST	`/events/create-orderbook`	`{"creatorName": "alice", "name": "X", "description": "...", "commission": 5, "commissionType": "ON_PURCHASE", "optionNames": ["Yes", "No"], "allowMint": true, "initial": 50, "d": 1}`	Create Event (Order Book).
POST	`/chat`	`{"userName": "alice", "message": "hi"}`	Send a chat message.
GET	`/chat`	-	Chat messages.

The browser sends each request to `http://localhost:5173/guessmarket/...`, and the web client's server forwards it unchanged to `http://localhost:8080/guessmarket/...` - the same base URL the JavaFX client uses.

Every response is JSON. A failed request returns `{"error": "..."}` with HTTP 400 for a validation error or HTTP 500 for an unexpected one, and the web client shows that message in its error dialog. `commissionType` is `ON_PURCHASE` or `ON_CLOSE`; `side` is `BUY` or `SELL`.

Design Choices and Assumptions

React and Vite, in plain JavaScript

React builds the screens: each second, it compares the new data from the server with what is on screen and updates only what actually changed, so nothing flickers and nothing you are typing is wiped. (The JavaFX client needed dedicated patch-in-place code for the same effect.) Vite runs the project and forwards the API calls, as described above. The code is plain JavaScript rather than TypeScript, as the exercise describes HTML, CSS and JS.

Vite 6 rather than the newest version

The newest Vite requires Node.js 20.19 or newer. Vite 6 runs on Node.js 18, 20 and 22+, so a grader with a somewhat older Node.js still gets a working client. It was also verified to work when npm skips package install scripts, which newer npm versions may do.

Live updates every second, like the JavaFX client

The client asks the server for the full current picture - events, users, chat, and the details of whatever event is open - once a second, the same pull approach and rate as the JavaFX client (the specification allows up to two seconds). A new request is only sent once the previous answer has arrived, so a busy server never gets a pile-up of overlapping requests.

A login lasts for the browser tab

The logged-in name is kept in the tab's session storage, so refreshing the page keeps you logged in - otherwise a refresh would lock you out, because registering the same name again is rejected as already taken. Closing the tab ends that login; every tab is a separate user.

The Account tab opens on your own page

The JavaFX client starts the Account tab with no user selected. In the browser it opens on your own account, since your balance and deposit are what you need first after logging in.

Number fields behave like the JavaFX Spinners

Quantity, price, deposit amount and the Create Event numbers have up/down arrows (hold to repeat, or use the arrow keys) and the same ranges and defaults as in the JavaFX client. As there, a typed value outside the range is clamped to the nearest legal value, and anything that is not a number reverts to the last valid value.

Same look as the JavaFX client

Same colors, pills, cards, error dialog, and per-user name colors (computed the same way, so a user has the same color in both clients). Two deliberate differences, for readability in a browser: the selected row in a list is a stronger blue, and alternating row stripes are slightly darker.

Server messages shown as-is

Every rule (positive quantities, price limits, enough shares to sell, unique event names, and so on) is enforced by the server, and its own message is shown in the error dialog - the web client does not duplicate the rules. Controls that do not apply to you (someone else's account, an event you are not the Market Maker of) are disabled and explain why on hover, with the same wording as the JavaFX client.

Main Files

Everything is in `web-client` (the repository's `guess-market-web` folder):

File	Role	JavaFX counterpart
`run.bat`	Checks Node.js, installs, starts the client	`run.bat`
`package.json`	Libraries (React, Vite) and the start command	Jars on the classpath
`vite.config.js`	Port 5173 and forwarding `/guessmarket` to Tomcat	-
`index.html`, `src/main.jsx`	The page and its entry point	`Main`
`src/App.jsx`	Login vs main screen; tabs; polling; error dialog	`MainController` (shell)
`src/api.js`	Every call to the server	`ApiClient`
`src/usePoll.js`	Ask the server again one second after each answer	The polling `Timeline`
`src/Login.jsx`	Login screen	`LoginController`, `login.fxml`
`src/EventsTab.jsx`	Filters and the event list	Events tab
`src/EventDetail.jsx`	LMSR and Order Book event views, read-only or with trading	`LmsrView`, `OrderBookView`
`src/AccountTab.jsx`	Users, account, deposit, history, participation, trading	Account tab
`src/CreateEventDialog.jsx`	Create Event form	Create Event dialog
`src/ChatTab.jsx`	Chat	Chat tab
`src/NumberSpinner.jsx`	Number field with arrows and clamping	`Spinner`
`src/ErrorDialog.jsx`	The error window	`showError`
`src/app.css`	All styling, using the JavaFX client's color tokens	`app.css`

Working with AI

Which tool did you choose, and why? Did you try other tools?

I used Claude Code, running inside the Antigravity editor - the same AI tool I worked with throughout the course. It works directly inside my project: it read my JavaFX client's code and my server's endpoints itself, so the web client was built to match what already existed rather than from my descriptions of it, and it could run the server, start the web client and test it in a real browser on its own.

For high-level structure and planning discussions I also used ChatGPT, but all of the actual development was done with Claude Code. I did not try GitHub Copilot for this exercise.

Were there areas where the AI did not "deliver the goods" and you had to step in as a programmer?

Yes - mainly in judgment calls, and in details only a person looking at the screen notices:

- Its first recommendation (React with a separate Node.js process) came without explaining why it was needed. I kept asking - why can't the browser just call my server on port 8080? - until I understood the browser's same-origin rule and why a proxy was the only option that keeps the server untouched. Only then did we choose the stack.
- Comparing it head-to-head with my JavaFX client, I noticed the number fields had no up/down arrows and handled bad input inconsistently (-3 and 0 gave two different error messages), unlike the JavaFX Spinners. I had it match the JavaFX behavior exactly.
- I noticed the alternating row shading was too faint and that a selected row looked like just another stripe, and asked for a clearer selection color.
- Scope decisions were mine: leave out XML upload, and add Create Event and Chat even though they were not required.

Did you run into a bug or requirement that the AI simply could not develop?

No - nothing it ultimately could not build. What it could not do on its own was judge how things look and feel; that needed me.

On its own initiative it also caught problems I would not have thought of: the newest version of its build tool would not run on older Node.js versions, so it switched to one that does and made `run.bat` check the version first; and it made the client say clearly when Tomcat is not running instead of showing a raw error. When its own automated test failed twice, both failures turned out to be mistakes in the test, not in the client - which it worked out correctly from screenshots.

How much of this client was developed by the AI alone, and how much needed your intervention and understanding?

Practically all of the code was written by the AI - I estimate over 95%; I did not type the code myself. But it was far from "just prompts": every decision went through me - the architecture, the framework, what to include and leave out, and how it should look and behave compared to my JavaFX client.

I made sure I understood how the parts connect (the proxy, React redrawing only what changed, polling every second, which endpoints are used) before moving on, tested each step myself in the browser, and sent things back when they did not match. In short: the AI wrote it; I directed, reviewed and tested it.

How long did it take? How long would it have taken without AI?

About two and a half hours in one evening - from installing Node.js to a tested, packaged client with its readme, including the discussion before any code was written. Without AI I estimate one to two weeks: learning React and the browser's rules from scratch, then building three screens with trading, and debugging.

Did you have prior front-end experience? Did it help?

Only a little HTML and CSS - no JavaScript frameworks. It helped a bit with reading the layout and styling, but React, Vite, Node.js and the same-origin/CORS rule were all new to me; I learned each one by asking the AI to explain it at my level before building on it.

Compared with the rest of the course, was this AI-guided way of working fun?

Yes. Seeing a working web version of my app appear in one evening - matching my JavaFX client and talking to the server I built in Exercise 3 - was genuinely fun. What kept it satisfying was staying hands-on: insisting on understanding why things work, catching what the AI missed, and making the decisions myself.

Exercises 1-3 taught me more about Java itself; this one taught me how to direct and check an AI's work - a different skill, and a useful one.